REWIND 言語リファレンス

対象は REWIND 3.6.0（言語バージョン `3.6.0`）です。言語の構文は 3.0.0 から変わっておらず、3.x の各版で増えたのは標準ライブラリとツールです。2.0.0 以前の言語（トップレベルに文を並べる書き方など）は、`rewind.toml` で言語バージョンを指定したときだけ使えます。この文書では扱いません。

この文書は **言語そのもの** の仕様書です。次のものは別の文書にあります。

知りたいこと	参照先
インストールと最初の実行	getting-started.md
標準ライブラリの各 module（関数、型、費用、失敗条件）	libraries/README.md、SDK の `share/rewind/doc/std/`
ネイティブ GUI	gui.md
デバッガ（DAP）	debugger.md
版ごとの変更点と契約	REWIND_v3.0.md など `REWIND_v*.md`
章立てで読む入門書	book/index.html（PDF 版）

目次

この文書の読み方

1. REWIND の考え方

2. プログラムを実行する

- 2.1 プログラムの形
- 2.2 コマンド
- 2.3 プロジェクトとマニフェスト

3. 字句

- 3.1 空白とコメント
- 3.2 識別子
- 3.3 整数と浮動小数点数のリテラル
- 3.4 文字列リテラル
- 3.5 文字列補間

4. 型

- 4.1 基本の型
- 4.2 複合型
- 4.3 型エイリアス

5. 式と演算子

- 5.1 演算子の優先順位
- 5.2 整数の算術

5.3 浮動小数点数の算術
5.4 比較と等価
5.5 文字列の連結
5.6 ブロック式、if 式、match 式
5.7 コレクションの式
5.8 タプル
5.9 メンバー呼び出し

6. 文と制御構造

6.1 束縛: let と var
6.2 定数: const
6.3 if と else
6.4 while
6.5 for
6.6 return
6.7 match とパターン
6.8 if let と let ... else

7. 関数

7.1 宣言
7.2 引数
7.3 クロージャ
7.4 関数型と高階関数
7.5 イテレータ

8. 型を定義する

8.1 struct: 変更できるデータ
8.2 record: 変更できない共有用のデータ
8.3 enum: 選択肢を持つ型
8.4 ジェネリクス
8.5 trait
8.6 公開と非公開

9. 所有権と借用

9.1 二種類の値
9.2 変更できる値は代入でコピーされない
9.3 move: 所有権を移す
9.4 借用: & と &mut
9.5 freeze と thaw

10. エラー処理

10.1 二種類の失敗
10.2 Option と Result
10.3 panic と assert
10.4 テスト

11. module

11.1 import
11.2 公開範囲
11.3 module の中の状態

12. 効果

- 12.1 効果とは
- 12.2 効果の一覧
- 12.3 効果は呼び出し元へ伝わる
- 12.4 宣言と許可は別
- 12.5 ファイルの扱い

13. チェックポイントと公開

- 13.1 三つの層
- 13.2 構文
- 13.3 revert
- 13.4 publish
- 13.5 begin
- 13.6 resume
- 13.7 branch
- 13.8 外部領域

14. 並行処理

- 14.1 async fn と Task
- 14.2 失敗、取り消し、時間切れ
- 14.3 チャネル
- 14.4 TaskGroup

15. 後始末と組み込みの入出力

- 15.1 defer
- 15.2 using
- 15.3 組み込みの入出力
- 15.4 文字列のメソッド
- 15.5 List と Map の組み込みメソッド

16. 実行予算と制限

- 16.1 予算の種類
- 16.2 ソースの中で指定する: runtime
- 16.3 そのほかの制限

17. 診断と記録

- 17.1 エラーの表示
- 17.2 終了コード
- 17.3 機械可読な診断
- 17.4 記録と再生
- 17.5 仕事量の内訳
- 17.6 デバッガとエディタ

18. 標準ライブラリの概観

付録 A. 構文の一覧

付録 B. 言語バージョンの違い

付録 C. 既知の制約

この文書の読み方

コード例は、特に断りがなければ **そのまま実行できる完全なプログラム** です。`main.rw` に保存して `rewind run main.rw` で動きます。例の下に `text` ブロックがあれば、それが標準出力です。エラーになる例は、起きるエラーのコードを添えています。

この文書のコード例は、リポジトリのテスト（`language_reference_examples_execute_as_documented`）が実際に実行し、出力と失敗の種類を確かめています。説明と実装がずれたら、テストが失敗します。

用語は次のように使います。

用語	意味
束縛	名前と値の結び付き。`let` と `var` で作る
所有者（owner）	変更できる値（`List`、`Map`、`struct` など）を保持する束縛
共有値（Share）	自由にコピーして共有できる値（`Int`、`String`、`record` など）
チェックポイント	計算の状態に付ける名前付きの保存点
保留中（pending）	まだ外部へ確定していない出力や書き込み
公開（publish）	保留中の入出力を外部へ確定すること
効果（effect）	関数が外部に対して行う操作の種類（出力、ファイル読み取りなど）

1. REWIND の考え方

REWIND は、計算の途中状態を **保存して巻き戻せる** プログラミング言語です。次の三つの層を区別すると、言語全体が理解しやすくなります。

1. **計算状態**: 変数、`List` や `Map` の中身、実行中のタスクなど、プログラムの中にある状態。チェックポイントで保存でき、`revert` で戻せます。
2. **保留中の入出力**: `Out.println` や `File.writeText` の結果。実行してもすぐには外へ出ず、保留されます。これも `revert` で取り消せます。
3. **公開済みの結果**: `publish` で外部へ確定した出力や書き込み。取り消せません。

REWIND

```
fn main()->Int {
    var score = 10;
    commit before;       // 計算状態を before として保存する
    score = 99;
    Out.println(score);  // 保留中になる
    publish;             // 99 を外へ確定する
    revert before;       // score は 10 に戻る。公開済みの 99 は取り消せない
    Out.println(score);
    return 0;
}
```

出力

```
99
10
```

`publish` を省くと、`99` は保留中のまま `revert` で捨てられ、`10` だけが出力されます。

REWIND には、このほかに次の設計上の特徴があります。

- **効果を宣言する**: 外部へ働きかける関数は、何をするかを `effects {...}` で宣言します。宣言していない操作はコンパイル時に拒否されます（第 12 章）。
- **所有権を検査する**: 変更できる値の複製は暗黙には起きません。`move`、借用（`&`、`&mut`）、`freeze` で扱いを明示します（第 9 章）。
- **実行は決定的**: タスクの切り替えを含め、同じ入力からは同じ結果になります。外部から得た値は記録され、再生できます（第 17 章）。
- **予算がある**: 実行ステップ、ネイティブ処理量、履歴メモリには上限があり、超えると実行が止まります（第 16 章）。

2. プログラムを実行する

2.1 プログラムの形

ソースファイルには **宣言だけ** を書きます。宣言とは `fn`、`async fn`、`struct`、`record`、`enum`、`type`、`trait`、`impl`、`const`、`import` です。実行される文は関数の中にだけ書けます。実行は `fn main` から始まります。

REWIND

```
fn main()->Int {
    Out.println("Hello, REWIND!");
    return 0;
}
```

出力

```
Hello, REWIND!
```

`main` は `Int` を返します。`0` は正常終了です。それ以外の値はプロセスの終了コードになり、標準エラーに `ApplicationExit<値>` が出ます。

`main` が戻ると、保留中の出力が自動的に公開されます。戻り値が 0 でなくても公開されます。`main` の中で `panic` やエラーが起きたときは公開されません。途中で外へ出したいときは、`publish;` を書きます。

トップレベルに文を書くと、実行前に拒否されます。

エラーになる例

```
let x = 1;
fn main()->Int { return 0; }
```

`fn main` が無いプログラムは実行できません（`rewind check` での検査は可能です）。

2.2 コマンド

コマンド

```
rewind run main.rw          # ソースを検査して実行する
rewind compile main.rw      # main.rwc を作る（実行はしない）
rewind run main.rwc         # 成果物を実行する。ソースは不要
```

`rewindc main.rw` は `rewind compile main.rw` と同じ、`rewind main.rwc` は `rewind run main.rwc` と同じです。単体のファイルを実行するのに、事前の `compile` やプロジェクトの設定は要りません。実行時には作業用の `.rewind` ディレクトリが作られます。

用途	コマンド
バージョン、ヘルプ	`rewind --version` 、`rewind --help` 、`rewind <コマンド> --help`
型・所有権・効果の検査だけ行う	`rewind check main.rw`
整形	`rewind fmt main.rw` （ `--check` で差分の検査のみ）
テストの実行	`rewind test --root DIR [--filter 名前]`
実行を記録する	`rewind run main.rw --record trace.json [--record-mode debug|compact]`
記録を再生する	`rewind replay trace.json --root DIR`
仕事量の内訳を見る	`rewind profile main.rw`
エディタ連携	`rewind lsp --root DIR` 、`rewind dap`
JSON 形式の診断	任意のコマンドに `--diagnostic-format json`

プログラムへ渡す引数は `--` の後ろに置きます（ `rewind run main.rw -- a b` ）。`--` より後ろの `--help` はプログラムの引数です。

`rewind fmt` は既存の改行を保ったままインデントを整えます。コードを一文ずつ改行し直す整形ではありません。

2.3 プロジェクトとマニフェスト

ファイルが複数あるプログラムや、権限・依存関係を固定したいプログラムは、ディレクトリに `rewind.toml` を置いてプロジェクトにします。

```
language = "3.6.0"
source_root = "src"
entry = "main.rw"
effects = "output,fileRead"
```

キー	意味
`language`	言語バージョン（必須）
`source_root`	ソースのあるディレクトリ
`entry`	実行を始めるファイル
`effects`	プログラムに許可する効果（カンマ区切り）
`[assets]`	実行に同梱するファイル
`[dependencies]` ほか	依存パッケージ。`rewind update` が `rewind.lock` を書く

マニフェストが近くにあるときは、その `language` と `effects` が使われ、ロックファイルの検証も行われます。`rewind update` は依存関係を解決して `rewind.lock` を更新します。依存パッケージの署名と信頼の仕組みは REWIND_v1.0.md 以降の各版の文書にあります。

3. 字句

3.1 空白とコメント

空白と改行は区切りでしかなく、意味を持ちません。文は `;` で終わり、ブロックは `{ ... }` で囲みます。

REWIND

```
fn main()->Int {
    // 行コメント
    /* ブロックコメント /* 入れ子にもできる */ */
    Out.println(1);
    return 0;
}
```

出力

```
1
```

3.2 識別子

識別子は英字、数字、`_` で構成し、数字では始められません。Unicode の文字も使えます（`let 変数 = 1;`）。大文字と小文字は区別されます。慣習として、型・trait・enum の variant は大文字で始め、関数・変数は小文字で始めます。

次の語は名前として使えません: `if`、`match`、`move`、`spawn`、`await`、`true`、`false`、`None`、`begin`（チェックポイントの予約名）。

`fn`、`let`、`for`、`struct`、`commit` など、そのほかのキーワードは文の先頭など決まった位置でだけ意味を持つため、名前に使えてしまいます。紛らわしいので使わないでください。

3.3 整数と浮動小数点数のリテラル

REWIND

```
fn main()->Int {
    Out.println(1_000);   // 数字の間に _ を入れられる
    Out.println(0xff);    // 16 進
    Out.println(0b1010);  // 2 進
    Out.println(0o755);   // 8 進
    Out.println(2.5);
    Out.println(1.5e3);   // 指数表記
    Out.println(2.5e-2);
    return 0;
}
```

出力

```
1000
255
10
493
2.5
1500
0.025
```

- 整数リテラルの範囲は符号付き 64 ビットです。範囲を超えると `IntegerOverflow` で拒否されます。`-9223372036854775808` は書けます。
- 小数点の前後には数字が必要です。`.5` や `5.` は書けません。
- 浮動小数点数は、整数部と小数部が整数値のときも `Out.println` では小数点なしで表示されます（`1.0` は `1`）。

3.4 文字列リテラル

文字列は二重引用符で囲み、UTF-8 として扱います。改行をそのまま含められます。

エスケープ	意味
`\n` `\r` `\t`	改行、復帰、タブ
`\"` `\\`	引用符、バックスラッシュ
`\0`	NUL 文字
`\u{754c}`	Unicode のコードポイント（1〜6 桁の 16 進数）

上記以外のエスケープ（`\q` など）はエラーです。文字を表す単独のリテラル（`'a'`）や、エスケープを無効にする raw 文字列はありません。

3.5 文字列補間

`${式}` で、式の値を文字列に埋め込めます。式には `Int`、`Float`、`Bool`、`String` の値が使えます。`$${` そのものを書きたいときは `$$${` とします。`${}` の中に文字列リテラルは書けません。

REWIND

```
fn main()->Int {
    let name = "REWIND";
    let n = 3;
    Out.println("${name}: ${n + 1} 件 ($${n})");
    return 0;
}
```

出力

```
REWIND: 4 件 (${n})
```

4. 型

4.1 基本の型

型	意味
`Int`	符号付き 64 ビット整数
`Float`	IEEE 754 の 64 ビット浮動小数点数。NaN と無限大を表せる
`Bool`	`true` 、`false`
`String`	UTF-8 の文字列
`Bytes`	バイト列。`Bytes("text")` で UTF-8 のバイト列を作る
`Unit`	値は `()` だけ。戻り値が無いことを表す
`Never`	決して戻らない式（ `panic` など）の型

`Int` と `Float` を混ぜた演算は型エラーです。暗黙の型変換は一切ありません。変換は明示します。

REWIND

```
fn main()->Int {
    let n: Int = 3;
    let x: Float = 1.5;
    // n + x はエラー。変換して合わせる
    match n.toFloatChecked() {
        Ok(f) => { Out.println(f + x); },
        Err(_) => { Out.println("変換できない"); }
    }
    Out.println(x.toIntChecked());    // 小数部があるので失敗する
    return 0;
}
```

出力

```
4.5
Err(InexactOrOverflow)
```

4.2 複合型

型	意味	例
`List<T>`	要素の型が `T` の可変の列	`List<Int>()`
`Map<K,V>`	キーと値の対応	`Map<String,Int>()`
`Option<T>`	値がある（ `Some(x)` ）か無い（ `None` ）か	

型	意味	例
`Result<T,E>`	成功（`Ok(x)`）か失敗（`Err(e)`）か	
`(A,B)`	タプル。要素は `.0`、`.1` で読む	`(1, "a")`
`Frozen<T>`	変更できない共有用の複製	`freeze(xs)`
`Secret<T>`	診断や記録で値を伏せるための型	`secret("pw")`
`Task<T>`	非同期関数の実行結果の予約	第 14 章
`Channel<T>`	タスク間でやり取りする通路	第 14 章
`fn(A)->B effects {...}`	関数の型	第 7 章

型引数は `<` `>` で囲みます。型を明示するときは `let x: Int = 1;` のように書きます。推論できる場合は省略できます。

4.3 型エイリアス

REWIND

```
type Score = Int;

fn main()->Int {
    let s: Score = 3;
    Out.println(s + 1);
    return 0;
}
```

出力

```
4
```

エイリアスは別名であり、新しい型は作りません。`Score` と `Int` は互いに代入できます。

5. 式と演算子

5.1 演算子の優先順位

弱い順に並べます。同じ行の演算子は左から結合します。括弧 `( )` で順序を明示できます。

順位	演算子	備考
1	`||`	論理和。左が `true` なら右を評価しない
2	`&&`	論理積。左が `false` なら右を評価しない
3	`==` `!=`	等価
4	`<` `<=` `>` `>=`	大小比較
5	`+` `-`	加減算、文字列の連結
6	`*` `/` `%`	乗除算、剰余
7	`!` `-`（前置）	論理否定、符号反転
8	`.` `[ ]` `( )` `?`（後置）	メンバー呼び出し、添字、関数呼び出し、エラーの伝播

ビット演算子（`&`、`|`、`^`、`<<`、`>>`）はありません。ビット操作は標準ライブラリの `std.bits` を使います。範囲 `a..b` は演算子ではなく、`for` 文と `match` のパターンの中でだけ書けます。

REWIND

```
fn main()->Int {
    Out.println(1 + 2 * 3);
    Out.println((1 + 2) * 3);
    Out.println(true || false && false);
    Out.println(-(-3));
    Out.println(!true);
    return 0;
}
```

出力

```
7
9
true
3
false
```

5.2 整数の算術

`Int` の `+ - * / %` は **検査付き** です。結果が 64 ビットの範囲を超えると `IntegerOverflow`、0 で割ると `DivisionByZero` で実行が止まります。丸めや折り返しは起きません。

- 除算は 0 に向かって切り捨てます（`-7 / 2` は `-3`）。
- 剰余の符号は割られる数に従います（`-7 % 3` は `-1`、`7 % -3` は `1`）。

REWIND

```
fn main()->Int {
    Out.println(7 / 2);
    Out.println(-7 / 2);
    Out.println(-7 % 3);
    Out.println(7 % -3);
    return 0;
}
```

出力

```
3
-3
-1
1
```

エラーになる例

```
fn main()->Int {
    let max = 9223372036854775807;
    Out.println(max + 1);
    return 0;
}
```

範囲を超える計算を `Result` で受けたいときは `std.integer` の検査付き関数を使います。それ以上の桁が必要なら `std.bigint` があります。

5.3 浮動小数点数の算術

`Float` の算術は IEEE 754 に従います。0 で割ると無限大や NaN になり、エラーにはなりません。NaN はそれ自身とも等しくありません。

REWIND

```
fn main()->Int {
    Out.println(1.0 / 0.0);
    Out.println(-1.0 / 0.0);
    let nan = 0.0 / 0.0;
    Out.println(nan == nan);
    Out.println(0.1 + 0.2);
    return 0;
}
```

出力

```
inf
-inf
false
0.30000000000000004
```

`Float` と `Int` を混ぜた演算は型エラーです（第 4.1 節）。

5.4 比較と等価

比較演算子 `< <= > >=` は `Int`、`Float`、`String` に使えます。文字列は UTF-8 のバイト列として辞書順に比べます。

`==` と `!=` の意味は、値の種類によって違います。

値	`==` の意味
`Int` `Float` `Bool` `String` `Bytes`	値が等しい
タプル、`record`、`enum`、`Option`、`Result`、`Frozen<T>`	中身が等しい
`List` `Map` `struct`（変更できる値）	**同一の値であるとき**だけ真。中身が同じでも別々に作った値は等しくない

変更できる値の中身を比べたいときは、`freeze` してから比べます（第 9.5 節）。

REWIND

```
fn main()->Int {
    let a = [1, 2];
    let b = [1, 2];
    Out.println(a == b);                    // 別の List なので false
    Out.println(freeze(a) == freeze(b));    // 中身を比べる
    Out.println((1, "x") == (1, "x"));
    Out.println(Some(3) == Some(3));
    return 0;
}
```

出力

```
false
true
true
true
```

`assert_eq(a, b)` も同じ規則で比べます。`List` どうしを `assert_eq` に渡すと、中身が同じでも失敗します。

5.5 文字列の連結

`+` は文字列を連結します。`+=` も使えます。数値などを文字列に混ぜるときは、文字列補間か `.format()` を使います。

REWIND

```
fn main()->Int {
    var s = "REWIND";
    s += " " + 3.format() + "!";
    Out.println(s);
    return 0;
}
```

出力

```
REWIND 3!
```

5.6 ブロック式、`if` 式、`match` 式

ブロックは最後の式（末尾に `;` が無いもの）を値として返します。`if` と `match` も値を持てます。分岐の値の型はそろっていなければなりません。

REWIND

```
fn main()->Int {
    let n = 5;
    let size = if n > 3 { "big" } else { "small" };
    let doubled = { let t = n * 2; t + 1 };
    let word = match n { 1 => "one", 2 => "two", _ => "many" };
    Out.println(size);
    Out.println(doubled);
    Out.println(word);
    return 0;
}
```

出力

```
big
11
many
```

`else` の無い `if` の値は `Unit` です。型が合わない分岐はエラーになります。

エラーになる例

```
fn main()->Int {
    let x = if true { 1 } else { "a" };
    return 0;
}
```

5.7 コレクションの式

`[ ]` は `List`、`{ キー: 値 }` は `Map` を作ります。要素の型は最初の要素で決まり、型の違う要素は拒否されます。空のコレクションは型を明示して `List<Int>()`、`Map<String,Int>()` と書きます。

REWIND

```
fn main()->Int {
    let xs = [10, 20, 30];
    let ages = {"ann": 31, "bob": 28};
    let grid = [[1, 2], [3]];
    xs[1] = 21;                       // 添字への代入
    Out.println(xs[1]);
    Out.println(grid[0][1]);
    Out.println(ages["ann"]);         // Map の添字は Option を返す
    Out.println(ages.len());
    return 0;
}
```

出力

```
21
2
Some(31)
2
```

- `xs[i]` は `xs.get(i)` と同じで、範囲外は `IndexOutOfBounds` で止まります。
- `m[k]` は `m.get(k)` と同じで、`Option` を返します。キーが無ければ `None` です。
- 文字列に添字は使えません。バイト位置や文字数は `String` のメソッドで扱います（第 15.4 節）。

5.8 タプル

`(a, b, c)` でタプルを作り、`.0`、`.1` で要素を読みます。入れ子は `t.1.0` のように続けます。分解して束縛することもできます。

REWIND

```
fn minmax(a: Int, b: Int) -> (Int, Int) effects {} {
    if a < b { return (a, b); }
    return (b, a);
}

fn main()->Int {
    let (low, high) = minmax(9, 4);
    let pair = minmax(1, 2);
    Out.println(low);
    Out.println(high);
    Out.println(pair.1);
    return 0;
}
```

出力

```
4
9
2
```

5.9 メンバー呼び出し

`値.名前(引数)` の形で、組み込み型のメソッドや、`trait` で定義したメソッドを呼びます。`struct` や `record` に直接メソッドを足す構文（`impl 型 { ... }`）はありません。メソッドは `trait` を通して定義します（第 8.5 節）。

関連関数（`型.new()` のような書き方）も無く、値を作る関数は普通の関数として書きます。

6. 文と制御構造

6.1 束縛: `let` と `var`

`let` で束縛した名前には再代入できません。`var` は再代入できます。型は推論されますが、`名前: 型` で明示もできます。

REWIND

```
fn main()->Int {
    let base = 10;
    var score: Int = base;
    score += 5;
    score -= 2;
    score *= 3;
    score /= 2;
    score %= 7;
    Out.println(score);
    return 0;
}
```

出力

```
5
```

エラーになる例

```
fn main()->Int {
    let x = 1;
    x = 2;
    return 0;
}
```

`let` の束縛が指す `List` などの **中身** は、所有権の規則を満たす限り更新できます。再代入できないのは「名前が指す値そのもの」です。

- 同じスコープで同じ名前を二度束縛することはできません（`duplicate binding`）。
- 内側のスコープでは同じ名前を束縛し直せます（外側の名前が隠れ、警告が出ます）。

束縛にはタプルの分解も使えます: `let (a, b) = pair;`。

6.2 定数: `const`

`const 名前: 型 = 式;` は、ファイルの先頭に書く名前付きの定数です。型の指定は省略できません。関数の中からも使え、`pub const` にすると他の module から読めます。

REWIND

```
const LIMIT: Int = 6 * 7;
const GREETING: String = "hello";

fn describe() -> String effects {} {
    return "${GREETING}, ${LIMIT}";
}

fn main()->Int {
    Out.println(describe());
    return 0;
}
```

出力

```
hello, 42
```

6.3 `if` と `else`

条件は `Bool` でなければなりません。`else if` で続けられます。

REWIND

```
fn grade(score: Int) -> String effects {} {
    if score >= 80 { return "A"; }
    else if score >= 60 { return "B"; }
    else { return "C"; }
}

fn main()->Int {
    Out.println(grade(90));
    Out.println(grade(65));
    Out.println(grade(10));
    return 0;
}
```

出力

```
A
B
C
```

6.4 `while`

REWIND

```
fn main()->Int {
    var i = 0;
    var total = 0;
    while i < 5 {
        i += 1;
        if i == 2 { continue; }   // 次の反復へ
        if i == 5 { break; }      // ループを抜ける
        total += i;
    }
    Out.println(total);
    return 0;
}
```

出力

```
8
```

`break` と `continue` は、いちばん内側のループに作用します。ラベル付きの `break` はありません。`break` が無い `while true { ... }` は、その後ろに `return` が無くても関数を終えられます。

6.5 `for`

`for 名前 in 式 { ... }` は二通りの形があります。

- **範囲** `a..b`: `a` 以上 `b` **未満** の整数を順に取ります。`a >= b` なら一度も実行しません。`..=` はありません。
- **コレクション**: `List`、`Frozen<List<T>>`、`xs.iter()` の結果などの要素を順に取ります。`Map` のキーは `m.keys()` で取り出します。

REWIND

```
fn main()->Int {
    for i in 0..3 { Out.println(i); }
    for name in ["a", "b"] { Out.println(name); }

    let ages = Map<String, Int>();
    ages.set("ann", 31);
    ages.set("bob", 28);
    for key in ages.keys() { Out.println(key); }   // キーの昇順
    return 0;
}
```

出力

```
0
1
2
a
b
ann
bob
```

`Map` のキーは常に昇順（`String` なら辞書順）で取り出されます。

6.6 `return`

`return 式;` で関数から戻ります。`Unit` を返す関数では `return;` と書けます。値を返す関数は、すべての経路で `return` に到達しなければならず、足りないとコンパイル時に `function returning Int needs a return on every path` と拒否されます。

6.7 `match` とパターン

`match` は値の形に応じて分岐します。各アームは `パターン => 式` か `パターン => { ... }` です。アームの間は `,` で区切ります。

REWIND

```
enum Shape {
    Circle(Int),
    Rect(Int, Int),
    Empty
}

fn area(s: Shape) -> Int effects {} {
    match s {
        Shape::Circle(r) => { return 3 * r * r; },
        Shape::Rect(w, h) if w == h => { return w * w; },   // ガード付き
        Shape::Rect(w, h) => { return w * h; },
        Shape::Empty => { return 0; }
    }
}

fn main()->Int {
    Out.println(area(Shape::Circle(2)));
    Out.println(area(Shape::Rect(3, 3)));
    Out.println(area(Shape::Rect(2, 5)));
    return 0;
}
```

出力

```
12
9
10
```

使えるパターンは次の通りです。

パターン	例	一致するもの
リテラル	`1`、`"a"`、`true`	等しい値
範囲	`0..5`	5 未満の 0 以上の整数
名前	`v`	どんな値にも一致し、`v` に束縛する
`_`	`_`	どんな値にも一致し、捨てる
variant	`Some(x)`、`Shape::Rect(w, h)`、`Ok(_)`	その variant。中身は入れ子のパターン
名前付きフィールド	`Msg::Move{x: a, y: b}`	フィールドを持つ variant
`record` / `struct`	`Point{x: a, y: b}`	各フィールド
タプル	`(1, s)`	要素ごとのパターン

アームには `if 条件` のガードを付けられます。

- **網羅性**: `enum`、`Option`、`Result` の `match` は、すべての場合を網羅しなければなりません。入れ子や組み合わせも検査され、ガード付きのアームは網羅の根拠になりません。漏れは `non-exhaustive match` として拒否されます。
- 先行するアームに含まれるだけのアームや、空の範囲は拒否されます。
- `|` による複数パターンの併記や、`x @ パターン` の束縛はありません。
- ガードの中から `publish` やチェックポイント操作は呼べません。

整数や文字列の `match` では、残りを受ける `_` が必要です。

REWIND

```
fn main()->Int {
    let n = 7;
    match n {
        0..5 => { Out.println("低"); },
        5..10 => { Out.println("中"); },
        _ => { Out.println("高"); }
    }
    return 0;
}
```

出力

```
中
```

6.8 `if let` と `let ... else`

値が特定の variant のときだけ処理したいときは `if let` を使います。

REWIND

```
fn main()->Int {
    let found: Option<Int> = Some(3);
    if let Some(v) = found {
        Out.println(v);
    } else {
        Out.println("なし");
    }
    return 0;
}
```

出力

```
3
```

`let パターン = 式 else { ... };` は、一致しなかったときに抜ける書き方です。`else` の中は `return`、`break`、`continue`、`panic` のどれかで終わらなければなりません。一致したときは、パターンの束縛が **後続の文の終わりまで** 使えます。

REWIND

```
fn firstOrZero(m: &Map<String, Int>) -> Int effects {} {
    let Some(v) = m.get("first") else { return 0; };
    return v;
}

fn main()->Int {
    let m = Map<String, Int>();
    Out.println(firstOrZero(&m));
    m.set("first", 9);
    Out.println(firstOrZero(&m));
    return 0;
}
```

出力

```
0
9
```

7. 関数

7.1 宣言

```
fn 名前<型引数>(引数: 型, ...) -> 戻り値の型 effects { 効果, ... } { 本体 }
```

- 引数と戻り値には型を書きます。戻り値を書かない関数は `Unit` を返します。ただし `pub` な関数は戻り値の型を省略できません（`-> Unit` と書きます）。
- `effects { ... }` はその関数が行う外部操作の宣言です。何もしない関数は `effects {}` と書きます（第 12 章）。`main` に限り省略できます。
- 関数は宣言の順に関係なく、ファイルのどこからでも呼べます。互いを呼び合う再帰も書けます。関数の中に関数を宣言することはできません。

REWIND

```
fn factorial(n: Int) -> Int effects {} {
    if n <= 1 { return 1; }
    return n * factorial(n - 1);
}

fn main()->Int {
    Out.println(factorial(5));
    return 0;
}
```

出力

```
120
```

本体が一つの式だけなら、`=` で短く書けます。戻り値の型は必須です。

REWIND

```
fn double(x: Int) -> Int = x * 2;
fn greet(name: String) -> String = "hi ${name}";

fn main()->Int {
    Out.println(double(21));
    Out.println(greet("REWIND"));
    return 0;
}
```

出力

```
42
hi REWIND
```

呼び出しの深さには上限があり、約 1,000 段を超えると `ExecutionBudgetExceeded: call depth` で止まります（第 16 章）。

7.2 引数

引数は位置で渡せます。宣言された関数（標準ライブラリを含む）には **名前付きの引数** も使えます。名前を使うと、順序は自由です。

引数には **既定値** を付けられます。既定値はリテラルか `None` で、既定値のある引数の後ろに既定値のない引数は置けません。

REWIND

```
fn box(width: Int, height: Int, label: String = "box") -> String effects {} {
    return "${label}: ${width * height}";
}

fn main()->Int {
    Out.println(box(3, 4));
    Out.println(box(height: 2, width: 5, label: "wide"));
    return 0;
}
```

出力

```
box: 12
wide: 10
```

所有権に関わる引数（変更できる値の `move`、借用の `&`、`&mut`）は 第 9 章にあります。

7.3 クロージャ

`|引数: 型, ...| -> 戻り値の型 { 本体 }` で名前の無い関数を作ります。型の指定は省略できず、本体は常に `{ }` で囲み、`return` で値を返します。外側の変数を読み書きできます。

REWIND

```
fn main()->Int {
    let add = |a: Int, b: Int| -> Int { return a + b; };
    Out.println(add(1, 2));

    var count = 0;
    let bump = || -> Unit { count += 1; };
    bump();
    bump();
    Out.println(count);
    return 0;
}
```

出力

```
3
2
```

クロージャが外側の変数をどう使えるかは、渡す先の関数型が決めます。タスクへ渡すクロージャには追加の制約があります（第 14 章）。

7.4 関数型と高階関数

関数を引数に取るには、関数型 `fn(引数の型) -> 戻り値の型 effects { ... }` を使います。名前付きの関数もクロージャも渡せます。渡す関数の効果は、受け取る型で許された範囲に収まっている必要があります。

REWIND

```
fn apply(f: fn(Int) -> Int effects {}, x: Int) -> Int effects {} {
    return f(x);
}

fn inc(n: Int) -> Int effects {} { return n + 1; }

fn main()->Int {
    Out.println(apply(inc, 4));
    Out.println(apply(|n: Int| -> Int { return n * 2; }, 4));
    return 0;
}
```

出力

```
5
8
```

7.5 イテレータ

`xs.iter()` で、`List` の **その時点の内容** を順に取り出すイテレータを作れます。元の `List` を後から変更しても、すでに作ったイテレータには影響しません。

メソッド	働き
`map(f)`	各要素を `f` で変換する
`filter(f)`	`f` が `true` を返す要素だけ残す
`fold(初期値, f)`	畳み込む
`take(n)`	先頭の `n` 個
`enumerate()`	`(添字, 要素)` のタプルにする
`zip(他)`	二つの列を組にする
`collect()`	`List` に集める

メソッド	働き
`next()`	次の要素を `Option` で返す

コールバックは同期的でなければならず、タスクの作成や `publish` を伴う関数は渡せません。

REWIND

```
fn main()->Int {
    let xs = [1, 2, 3, 4];
    let evens = xs.iter()
        .filter(|n: Int| -> Bool { return n % 2 == 0; })
        .map(|n: Int| -> Int { return n * 10; })
        .collect();
    Out.println(evens.len());
    Out.println(evens.get(1));
    Out.println(xs.iter().fold(0, |a: Int, b: Int| -> Int { return a + b; }));
    return 0;
}
```

出力

```
2
40
10
```

8. 型を定義する

8.1 `struct`: 変更できるデータ

`struct` は、フィールドを後から書き換えられるデータ型です。値を作るときは、フィールドの宣言順に引数を渡します。

REWIND

```
struct Counter { value: Int }

fn main()->Int {
    let c = Counter(1);
    c.value = 2;
    c.value += 5;
    Out.println(c.value);
    return 0;
}
```

出力

```
7
```

`struct` は「変更できる値」なので、所有権の規則に従います（第 9 章）。コピーはできず、`==` は同一の値かどうかを調べます。

8.2 `record`: 変更できない共有用のデータ

`record` は、作った後に変更できないデータ型です。フィールドの型はすべて **共有値**（第 9.1 節）でなければなりません。代わりに、自由にコピーして共有でき、`==` は中身を比べます。

REWIND

```
record Point { x: Int, y: Int }

fn main()->Int {
    let p = Point(3, 4);
    let q = p;                          // コピーしてよい
    Out.println(p.x + q.y);
    Out.println(p == Point(3, 4));
    return 0;
}
```

出力

```
7
true
```

エラーになる例

```
record Point { x: Int, y: Int }
fn main()->Int {
    let p = Point(3, 4);
    p.x = 1;
    return 0;
}
```

`List` のような変更できる値を `record` に持たせたいときは、`Frozen<List<T>>` にします（第 9.5 節）。

REWIND

```
record Config { names: Frozen<List<String>>, limit: Int }

fn main()->Int {
    let c = Config(freeze(["a", "b"]), 3);
    Out.println(c.names.len());
    return 0;
}
```

出力

```
2
```

`record` や `struct` を作るときの引数は位置で渡します。`Point(x: 1, y: 2)` のような名前付きの引数や、`Point { x: 1 }` のような波括弧の構築式は、`record` と `struct` にはありません。フィールドの取り出しには、パターンも使えます。

REWIND

```
record Point { x: Int, y: Int }

fn main()->Int {
    let p = Point(3, 4);
    match p {
        Point{x: a, y: b} => { Out.println(a * b); }
    }
    return 0;
}
```

出力

```
12
```

8.3 `enum`：選択肢を持つ型

`enum` の各 variant は、値を持たないもの、位置で値を持つもの、名前付きのフィールドを持つものを混ぜて宣言できます。値は `enum名::variant` で作ります。

REWIND

```
enum Message {
    Quit,
    Write(String),
    Move{x: Int, y: Int}
}

fn describe(m: Message) -> String effects {} {
    match m {
        Message::Quit => { return "quit"; },
        Message::Write(text) => { return "write " + text; },
        Message::Move{x: a, y: b} => { return "move ${a},${b}"; }
    }
}

fn main()->Int {
    Out.println(describe(Message::Quit));
    Out.println(describe(Message::Write("hi")));
    Out.println(describe(Message::Move{x: 1, y: 2}));
    return 0;
}
```

出力

```
quit
write hi
move 1,2
```

`match` では、すべての variant を扱わなければなりません（第 6.7 節）。`enum` どうしの `==` は、variant と中身が等しいかを調べます。

8.4 ジェネリクス

型や関数は、型引数を取れます。`<T>` で宣言し、必要なら `T: 制約` で能力を要求します。複数の制約は `+` でつなぎ、`where` 節でも書けます。

制約	意味
`Share`	共有値であること（コピーして共有できる）
`Send`	タスクへ渡せること
`Ord`	大小比較（`cmp`）ができ、`Map` のキーや並べ替えに使えること
`Eq`	等しさを比べられること
`Hash`	ハッシュ値を計算できること
`Display`	文字列に表示できること

REWIND

```
struct Stack<T> { items: List<T> }
enum Tree<T> { Leaf, Node(T) }

fn largest<T>(a: T, b: T) -> T effects {} where T: Ord + Share {
    if a.cmp(b) >= 0 { return a; }
    return b;
}

fn main()->Int {
    let s = Stack<Int>(List<Int>());
    s.items.add(1);
    let t: Tree<Int> = Tree::Node(5);
    match t {
        Tree::Leaf => {},
        Tree::Node(v) => { Out.println(v); }
    }
    Out.println(s.items.len());
    Out.println(largest(3, 9));
    Out.println(largest("a", "b"));
    return 0;
}
```

出力

```
5
1
9
b
```

型引数は、引数や期待する型から決まるときは省略できます。決まらないときは `List<Int>()`、`Stack<Int>(...)` のように明示します。ジェネリックな `record` では、型引数に `Share` 制約が必要です（`record Box<T: Share> { value: T }`）。

8.5 `trait`

`trait` は、型が提供するメソッドの約束です。`impl トレイト for 型` で実装します。メソッドの第一引数 `self` の型は `&Self`（読むだけ）、`&mut Self`（変更する）、`Self`（所有権を受け取る）から選びます。実装は `trait` が宣言したメソッドをすべて定義しなければならず、同じ型に同じ `trait` を二度実装することはできません。

REWIND

```
trait Shape {
    fn area(self: &Self) -> Int effects {};
    fn describe(self: &Self) -> String effects {} {          // 既定の実装
        return "面積 " + self.area().format();
    }
}

record Square { side: Int }

impl Shape for Square {
    fn area(self: &Square) -> Int effects {} { return self.side * self.side; }
}

fn total<T: Shape>(item: &T) -> Int effects {} { return item.area(); }

fn main()->Int {
    let sq = Square(3);
    Out.println(sq.area());
    Out.println(sq.describe());
    Out.println(total(&sq));
    return 0;
}
```

出力

```
9
面積 9
9
```

`trait` のメソッドは `値.メソッド()` の形で呼びます。`Int` のような組み込みの型にも実装できます。独自の型を `Map` のキーにしたり `Ord` で比べたりするには、`Ord` を実装します（引数は `Self` で受けます）。

REWIND

```
record Version { major: Int }

impl Ord for Version {
    fn cmp(self: Self, other: Self) -> Int effects {} {
        if self.major < other.major { return -1; }
        if self.major > other.major { return 1; }
        return 0;
    }
}

fn main()->Int {
    let m = Map<Version, String>();
    m.set(Version(2), "two");
    m.set(Version(1), "one");
    Out.println(m.len());
    Out.println(Version(2).cmp(Version(1)));
    return 0;
}
```

出力

```
2
1
```

`struct` や `record` に直接メソッドを定義する構文はありません。メソッドを持たせたいときは `trait` を定義して実装します。

8.6 公開と非公開

型・関数・定数・`trait` に `pub` を付けると、他の module から使えます。`struct` と `record` のフィールドに `private` を付けると、その module の外から読み書き・構築・分解ができなくなります。値を作る関数と、読み書きする関数を公開して使わせます（第 11 章）。

9. 所有権と借用

REWIND は、変更できる値を **暗黙にコピーしません**。誰がその値を持っているかを型検査が追跡し、同じ値を二か所から同時に変更してしまう誤りを、実行前に拒否します。

9.1 二種類の値

種類	例	扱い
共有値（Share）	`Int` 、`Float` 、`Bool` 、`String` 、`Bytes` 、`Unit` 、`record` （フィールドがすべて共有値のもの）、`Frozen<T>` 、タプル・`Option` の共有値	自由にコピー・共有できる
変更できる値（所有者）	`List` 、`Map` 、`struct` 、イテレータ、ファイルハンドルなどの資源	一か所が所有する。コピーできない

束縛が `let` か `var` かは、値の所有権とは別の話です。`let xs = [1];` でも、`xs` に `add` で要素を足せます。`let` が禁じるのは、`xs` に別の値を代入し直すことだけです。

9.2 変更できる値は代入でコピーされない

エラーになる例

```
fn main()->Int {
    let a = [1];
    let b = a;      // コピーも共有もできない
    return 0;
}
```

別の名前で持ちたいときは、次のどれかを選びます。

1. 所有権を渡す: `move`
2. 一時的に貸す: `&` （読むだけ）、`&mut` （変更する）
3. 変更できない複製を作って共有する: `freeze`

9.3 `move` : 所有権を移す

`move 式` は、所有権を新しい持ち主へ渡します。渡した後の元の名前は、使うとエラーです。

REWIND

```
fn count(xs: List<Int>) -> Int effects {} {
    return xs.len();
}

fn main()->Int {
    let xs = [1, 2, 3];
    Out.println(count(move xs));
    return 0;
}
```

出力

```
3
```

エラーになる例

```
fn count(xs: List<Int>) -> Int effects {} { return xs.len(); }

fn main()->Int {
    let xs = [1, 2, 3];
    let n = count(move xs);
    Out.println(xs.len());   // 渡した後は使えない
    return 0;
}
```

`move` を付けずに変更できる値を関数へ渡すと、`passing a mutable owner requires move or freeze` と拒否されます。

9.4 借用: `&` と `&mut`

関数に所有権を渡さず、一時的に貸すのが借用です。引数の型を `&T`（読むだけ）、`&mut T`（変更する）と宣言し、呼び出しでも同じ記号を付けます。

REWIND

```
fn total(values: &List<Int>) -> Int effects {} {
    var sum = 0;
    for i in 0..values.len() { sum += values.get(i); }
    return sum;
}

fn append(values: &mut List<Int>, n: Int) -> Unit effects {} {
    values.add(n);
}

fn main()->Int {
    let xs = List<Int>();
    append(&mut xs, 7);
    append(&mut xs, 8);
    Out.println(total(&xs));
    return 0;
}
```

出力

```
15
```

借用には次の規則があります。

- `&` の借用は同時にいくつでも持てますが、借用中に持ち主は変更できません。
- `&mut` の借用は同時に一つだけです。同じ値を二つの `&mut` で貸すことはできません。
- `&` の借用からは、値を変更できません。
- 借用を関数の戻り値にしたり、タスクへ渡したりすることはできません（`borrowed return types are not supported`）。

エラーになる例

```
fn bad(xs: &List<Int>) -> Unit effects {} {
    xs.add(1);
}
fn main()->Int { return 0; }
```

エラーになる例

```
fn f(a: &mut List<Int>, b: &mut List<Int>) -> Unit effects {} {}

fn main()->Int {
    let xs = [1];
    f(&mut xs, &mut xs);
    return 0;
}
```

メソッドの呼び出し（`xs.add(1)`、`xs.len()`）では、借用の記号は自動で付きます。

9.5 `freeze` と `thaw`

`freeze(値)` は、その時点の内容を **変更できない共有用の複製** `Frozen<T>` にします。`Frozen` は共有値なので、コピーも `record` のフィールドへの格納もできます。読み取り（`len`、`get`、`for`）はできますが、変更するとエラーです。`thaw(frozen)` は、元の `Frozen` と独立した、変更できる値を作ります。

REWIND

```
fn sum(xs: Frozen<List<Int>>) -> Int effects {} {
    var total = 0;
    for x in xs { total += x; }
    return total;
}

fn main()->Int {
    let snapshot = freeze([1, 2, 3]);
    Out.println(sum(snapshot));
    Out.println(sum(snapshot));      // 何度でも使える

    let copy = thaw(snapshot);
    copy.add(4);
    Out.println(copy.len());
    Out.println(snapshot.len());     // 元は変わらない
    return 0;
}
```

出力

```
6
6
4
3
```

エラーになる例

```
fn main()->Int {
    let s = freeze([1]);
    s.add(2);
    return 0;
}
```

`freeze` できない値もあります。関数値、ファイルや接続のような資源、深さが 64 を超えるものや循環するものです。`freeze` と `iter` が使う処理量と領域は、実行予算に計上されます（第 16 章）。

10. エラー処理

10.1 二種類の失敗

REWIND の失敗は、扱い方で二種類に分かれます。

- **回復できる失敗**: 入力の誤りや見つからない値など、呼び出し側が選択して処理すべきもの。`Option` または `Result` で返します。
- **致命的な失敗**: プログラムの誤りや資源の限度超過など。`panic`、`assert` の失敗、リストの範囲外アクセス、ゼロ除算、整数のオーバーフロー、実行予算の超過がこれにあたります。実行は止まり、診断が出力されます。関数の `Result` に自動では変換されません。

タスクの中で起きた致命的な失敗は、そのタスクを `await` した側に `TaskError` として届きます（第 14 章）。

10.2 `Option` と `Result`

`Option<T>` は `Some(値)` か `None`、`Result<T,E>` は `Ok(値)` か `Err(エラー)` です。`match`、`if let`、`let ... else` で取り出します。`unwrap` のような、失敗を無視して取り出すメソッドはありません。

`Result` には次のメソッドがあります。

メソッド	働き
`map(f)`	`Ok` の中身を変換する
`mapErr(f)`	`Err` の中身を変換する
`andThen(f)`	`Ok` の中身から次の `Result` を作る
`flatten()`	入れ子の `Result` を一つにする（`Err` の型が同じとき）

`式?` は、`Result` を返す関数の中で使えます。`Ok(v)` なら `v` を取り出し、`Err(e)` ならその関数から `Err(e)` を返します。

REWIND

```
fn parse(text: String) -> Result<Int, String> effects {} {
    return text.parseInt();
}

fn sum(a: String, b: String) -> Result<Int, String> effects {} {
    let x = parse(a)?;
    let y = parse(b)?;
    return Ok(x + y);
}

fn main()->Int {
    Out.println(sum("1", "2"));
    Out.println(sum("1", "x"));
    match sum("4", "5") {
        Ok(v) => { Out.println(v); },
        Err(e) => { Out.println(e); }
    }
    return 0;
}
```

出力

```
Ok(3)
Err(InvalidIntOrOverflow)
9
```

`?` は `Option` には使えません。`Option` は `match` や `let ... else` で処理します。

関数が返す `Err` の型は、`?` を使う関数の戻り値の `Err` の型と一致しなければなりません。型が違うときは `mapErr` で変換してから `?` を使います。

REWIND

```
fn lookup(key: String) -> Result<Int, String> effects {} {
    if key == "a" { return Ok(1); }
    return Err("missing");
}

fn code(key: String) -> Result<Int, Int> effects {} {
    let v = lookup(key).mapErr(|e: String| -> Int { return e.byteLen(); })?;
    return Ok(v);
}

fn main()->Int {
    Out.println(code("a"));
    Out.println(code("b"));
    return 0;
}
```

出力

```
Ok(1)
Err(7)
```

> **現状の制約**: `?` のエラー型の不一致は、現在のところ `Err` の経路を実際に通ったときの実行時エラー（`b returns Result<Unknown,String>, expected ...`）として検出されます。コンパイル時には拒否されないため、`mapErr` で型をそろえる書き方を習慣にしてください。

10.3 `panic` と `assert`

呼び出し	働き
`panic("メッセージ")`	実行を止める。型は `Never`
`assert(条件)`	偽なら `AssertionFailed` で止まる
`assert_eq(a, b)`	等しくなければ `AssertionFailed`（左右の値を表示する）。等しさは 第 5.4 節の規則

これらは入力検査のための仕組みではありません。利用者の入力など、起こりうる誤りには `Result` を返す関数を使います。`main` の中で止まったとき、保留中の出力は公開されません。

10.4 テスト

`test fn` で書いた関数は、通常の実行では呼ばれず、`rewind test` で実行されます。`assert` と `assert_eq` で検証します。

REWIND

```
fn double(x: Int) -> Int = x * 2;

test fn doubles() {
    assert_eq(double(4), 8);
}

fn main()->Int {
    Out.println(double(21));
    return 0;
}
```

出力

```
42
```

`rewind test --root DIR` はプロジェクトのテストを順に実行し、成功した名前を `ok 名前` で表示します。失敗するとそこで止まります。テストの中では `publish` は使えません。`--filter 名前` で絞り込めます。

11. module

11.1 `import`

`import` はファイルの先頭に書きます。別名（`as`）を付けて使うのが基本です。

REWIND

```
import std.number as number;
import std.text as text;

fn main()->Int {
    Out.println(number.integer("ff", 16));
    Out.println(text.length("日本語"));
    return 0;
}
```

出力

```
Ok(255)
3
```

書き方	意味
`import std.number as number;`	標準ライブラリの module を別名で使う
`import util as u;`	同じプロジェクトのファイル `util.rw` を別名で使う
`import util.{double, total};`	名前を選んで直接取り込む
`import util.{double as dbl};`	取り込む名前に別名を付ける

別名を付けない `import util;` では、`util` という名前は使えるようになりません。必ず `as` か `{ }` を使います。`import` の別名は、ファイルごとの名前です。

単体のファイルの実行では、標準ライブラリはコンパイラに内蔵されており、相対 import はそのファイルのあるディレクトリから探されます。プロジェクトでは `rewind.toml` の `source_root` から探します。

11.2 公開範囲

ファイルの中の宣言は、既定では他のファイルから見えません。`pub` を付けた宣言だけが公開されます。

公開できるものは、`fn`、`struct`、`record`、`enum`、`type`、`trait`、`const` です。公開する関数には戻り値の型が必要です。公開した `struct` のフィールドを `private` にすると、内部の表現を隠したまま、公開した関数を通して扱わせることができます。

```
// util.rw
pub struct Account { private total: Int }
pub fn open() -> Account effects {} { return Account(0); }
pub fn deposit(a: &mut Account, n: Int) -> Unit effects {} { a.total += n; }
pub fn balance(a: &Account) -> Int effects {} { return a.total; }
pub const LIMIT: Int = 99;

// main.rw
import util as u;
fn main()->Int {
    let a = u.open();
    u.deposit(&mut a, 5);
    Out.println(u.balance(&a));   // 5
    Out.println(u.LIMIT);         // 99
    // a.total は module の外から読めない
    return 0;
}
```

公開されていない関数を呼ぶと `unknown name` と報告されます。非公開フィールドを読むと、`field total is private to its module` です。

11.3 module の中の状態

module の最上位に書けるのは宣言だけで、`const` を除いて「状態」は持てません。import した関数は、呼び出し元の変数を名前では参照できません。

12. 効果

12.1 効果とは

効果（effect）は、関数が外の世界に対して行う操作の種類です。画面への出力、ファイルの読み書き、現在時刻の取得などがあります。REWIND では、関数が行う効果を `effects { ... }` で宣言します。宣言に無い効果を使うプログラムは、実行する前に拒否されます。

エラーになる例

```
fn greet() -> Unit effects {} {
    Out.println("hi");        // output を宣言していない
}

fn main()->Int {
    greet();
    return 0;
}
```

何もしない関数は `effects {}` と書きます。これは「この関数は外部と無関係で、同じ引数なら同じ結果を返す」という約束になり、巻き戻しや再生を安全にする土台です。

12.2 効果の一覧

効果	できること
`output`	`Out` 、`Err` への出力
`input`	`In` からの入力
`args`	`Args` による起動引数の読み取り
`env`	`Env` による環境変数の読み取り
`locale`	`Locale` による地域設定の読み取り
`clock`	`Time` による時刻の観測
`random`	`Random` による乱数
`tasks`	タスク、チャネルの作成と待機
`fileRead`	`File` の読み取り、`Directory` の一覧
`fileWrite`	`File` 、`Directory` の書き込み・作成・削除
`external`	`external { }` 領域での外部操作
`live`	`external live { }` 領域での、再利用しない物理操作

効果	できること
`network`	HTTP、TCP などの通信
`db`	SQLite、PostgreSQL
`gui`	ネイティブ画面

ここにない名前を宣言すると `unknown effect` です。

12.3 効果は呼び出し元へ伝わる

関数が呼ぶ関数の効果は、呼ぶ側も宣言しなければなりません。呼び出しの連鎖のどこかで宣言が足りないと、不足している箇所と原因の経路が `caused by` として表示されます。

REWIND

```
fn greet() -> Unit effects {output} {
    Out.println("hi");
}

fn twice() -> Unit effects {output} {
    greet();
    greet();
}

fn main()->Int effects {output} {
    twice();
    return 0;
}
```

出力

```
hi
hi
```

`main` だけは `effects` を省略でき、そのときは本体が必要とする効果が自動で求められます。

12.4 宣言と許可は別

関数の宣言は「何をするか」の表明です。それを実際に許すかどうかは、プログラムを **実行する側** が決めます。宣言だけでは何も許可されません。

実行のしかた	許可される効果
単体のファイルの `rewind run`	既定で `input`、`output`、`args`、`locale`、`random`、`tasks`
追加で許可する	`--allow-effects fileRead,fileWrite` のようにカンマ区切りで指定する
プロジェクト	`rewind.toml` の `effects`

`rewind compile` と `rewind run main.rwc` にも同じ指定が必要です。成果物に効果を記録しても、実行側の許可は別に要ります。許可が無いときは `effect gui is not allowed` のように拒否されます。

環境変数は、効果 `env` に加えて **名前ごとの許可** が必要です。`--allow-env 名前` で許可し、値を診断や記録に残したくないときは `--secret-env 名前` を使います。

12.5 ファイルの扱い

`File` と `Directory` のパスは、実行の root（単体のファイルではそのファイルのあるディレクトリ、プロジェクトでは `--root`）からの相対パスで、区切りには OS に関係なく `/` を使います。絶対パス、`..`、root の外を指すシンボリックリンクは `InvalidPath` で拒否されます。

読み取りの失敗（存在しない、パスが不正など）は `Result<T,FileError>` で返ります。

REWIND

```
fn main()->Int effects {fileRead, fileWrite, output} {
    File.writeText("state.txt", "saved");
    Out.println(File.readText("state.txt"));    // 保留中の書き込みも読める
    Out.println(File.readText("missing.txt"));
    Out.println(File.readText("../escape.txt"));
    return 0;
}
```

出力

```
Ok(saved)
Err(NotFound: missing.txt (file not found: missing.txt))
Err(InvalidPath: ../escape.txt (invalid path: ../escape.txt))
```

書き込みは保留中になり、`publish` か `main` の終了で実際のファイルに反映されます（第 13 章）。

13. チェックポイントと公開

13.1 三つの層

第 1 章で述べたとおり、実行中のものは次の三つの層に分かれます。

層	内容	`revert` で
計算状態	変数、`List` / `Map` の中身、タスク、イテレータの位置、仮想的なファイル、入力の読み取り位置、乱数の状態	戻る
保留中の入出力	`Out.println` や `File.writeText` の結果で、まだ外へ出ていないもの	取り消される
確定済みの結果	`publish` で外へ出た出力・書き込み	戻らない。再送もされない

そのため `commit` は「外へ確定する操作」ではありません。後で戻るための保存点を作るだけです。

13.2 構文

構文	意味
`commit 名前;`	計算状態と保留中の入出力を、その名前で保存する
`revert 名前;`	保存した状態へ戻す。実行位置は動かない
`resume 名前;`	状態に加え、実行位置も保存した位置へ戻す
`drop 名前;`	保存点を捨てて、保持している記憶を解放する
`branch 名前 { ... }`	試しに実行し、終わると親の状態へ戻る
`publish;`	保留中の入出力を外へ確定する

チェックポイントの名前は、変数や関数の名前とは別の名前空間にあります。同じ名前で二度 `commit` することはできません。存在しない名前への `revert` や `drop` はエラーです。`begin` は予約名です（第 13.5 節）。

13.3 `revert`

`revert` は変数や `List` の中身を保存時の状態に戻し、保存後に溜まった保留中の出力を捨てます。公開済みの出力は戻りません。

REWIND

```
fn main()->Int {
    let items = [1, 2];
    var n = 10;
    commit saved;

    items.add(3);
    n = 99;
    Out.println(n);          // 保留中のまま revert で消える

    revert saved;
    Out.println(items.len());
    Out.println(n);
    drop saved;
    return 0;
}
```

出力

```
2
10
```

`revert` は実行位置を変えません。次に実行されるのは `revert` の次の文です。そのため、保存点を作った関数の中で、`if` やループの内側から外側の保存点へ戻せます。保存点を作った後で終わった内側のスコープや、別の関数の呼び出しの中で作られた保存点へは戻せません（`InvalidContinuation`）。

エラーになる例

```
fn undo() -> Unit effects {} {
    revert outer;           // 呼び出し元で作った保存点には戻れない
}

fn main()->Int {
    commit outer;
    undo();
    return 0;
}
```

保存点の後に作った変数は、`revert` で消えます。

13.4 `publish`

`publish` は保留中の入出力を外へ出します。公開した後の `revert` で戻る範囲は、公開より後の変化だけです。

REWIND

```
fn main()->Int effects {output} {
    var score = 10;
    commit first;
    score = 99;
    Out.println(score);
    publish;                  // 99 が外へ出る
    revert first;
    Out.println(score);       // 10。これは保留中
    return 0;                 // main の終了で公開される
}
```

出力

```
99
10
```

公開は複数のファイルやストリームをまとめて一度に適用する保証はありません。適用の途中で失敗すると `PublishPartiallyApplied` になり、どこまで適用されたかが診断に出ます。外部の状態を確かめてから判断してください。同じプロセスでの盲目的な再公開は拒否されます。

`main` が戻ると、保留中の出力は自動で公開されます。`panic` や失敗で止まったときは公開されません。入力を待つ前に出力を見せたい対話型のプログラムは、`In.readLine()` の前に `publish;` を書きます。

13.5 `begin`

ユーザーのコードを実行する前に、`commit begin;` に相当する保存点が自動で作られます。`revert begin;` は、ここまでに作った変数やチェックポイントを含めて、すべてを初期状態に戻します（確定済みの結果は残ります）。`begin` は予約名なので、自分で `commit` や `drop` はできず、変数や関数の名前にも使えません。

`revert begin` で消えた変数は参照できなくなり、同じ名前で宣言し直せます。実行中の関数やブロックの枠は残るので、その後の `return` やブロックの終わりは使えます。メインタスクの中で、`branch` の外から使います。

13.6 `resume`

`resume 名前;` は、状態だけでなく実行位置も `commit 名前;` の直後へ戻します。状態も一緒に戻るので、状態だけを脱出条件にしたループは決して終わりません。そのときは実行予算を使い切って止まります。

エラーになる例

```
fn main()->Int {
    var n = 0;
    commit start;
    n += 1;                  // resume で n は 0 に戻る
    if n < 3 { resume start; }
    return 0;
}
```

脱出条件には、巻き戻らないものを使います。公開済みの結果、消費した予算、記録された外部の観測（第 13.8 節）がそれにあたります。

13.7 `branch`

`branch 名前 { ... }` は、ブロックの中を試しに実行し、終わると計算状態と保留中の入出力を実行前に戻します。条件ごとに別の方針を試し、結果だけを使うような場面で使います。ブロックの中で `return` すると、そこで関数から戻れます。

REWIND

```
fn main()->Int {
    var best = 0;
    branch trial {
        best = 100;                // 試した結果は残らない
        Out.println("試行中");     // 保留中の出力も捨てられる
    }
    Out.println(best);
    return 0;
}
```

出力

```
0
```

`branch` の中で `publish` した結果は、`branch` の終了でも戻りません。

13.8 外部領域

外部のサービスから得た値（現在の時刻、ネットワークの応答など）は、巻き戻した後に取り直すと別の値になりえます。REWIND は、そうした操作を `external { ... }` という **外部領域** に閉じ込めます。領域内の操作の結果は **記録** され、同じ保存点へ戻って実行し直したときは、記録された結果が再利用されます。

REWIND

```
import std.external as host;

fn main()->Int effects {external, clock, fileRead, fileWrite, output} {
    var stamp = 0;
    commit before;
    external {
        match host.millis() {
            Ok(n) => { stamp = n; },
            Err(_) => {}
        }
    }
    File.writeText("first.txt", stamp.format());
    publish;                        // 一度目の値をファイルに確定する
    revert before;                  // stamp は 0 に戻る
    external {
        match host.millis() {
            Ok(n) => { stamp = n; },
            Err(_) => {}
        }
    }
    match File.readText("first.txt") {
        Ok(text) => { Out.println(text == stamp.format()); },
        Err(_) => { Out.println("読めない"); }
    }
    return 0;
}
```

出力

```
true
```

二度目の `host.millis()` は時計を読み直さず、一度目に記録した値を返します。そのため、巻き戻す前後で同じ値になります。巻き戻す前の値を後から比べるには、確定済みのファイルのような、巻き戻らない場所に残す必要があります。

形	意味
`external { ... }`	結果を記録し、巻き戻した後は記録を再利用する
`external fresh { ... }`	常に新しい操作を実行し、その結果を記録する
`external live { ... }`	物理操作を実行する。巻き戻しや再生では再利用しない。`live` 効果の許可が必要

領域の中では、外部操作だけを行います。`await`、`publish`、`commit`、`revert`、`return`、`break`、`continue` は拒否されます（`ExternalBoundary`）。通信を領域内で始め、領域を出てから `await` で結果を受け取ります。データベースの書き込みなど、すでに外へ及んだ作用は `revert` で取り消せません。

14. 並行処理

14.1 `async fn` と `Task`

`async fn` は、呼ぶとすぐには実行されず、`Task<T>` を返します。タスクは `await` するか、`spawn` で開始すると実行されます。`await` は `Result<T, TaskError>` を返すので、関数自身の失敗とタスクの失敗を区別して扱えます。

REWIND

```
async fn compute(n: Int) -> Int effects {} {
    return n * 2;
}

fn main()->Int {
    let direct = compute(4);
    match await direct {
        Ok(v) => { Out.println(v); },
        Err(_) => { Out.println("失敗"); }
    }

    let started = spawn compute(21);
    match await started {
        Ok(v) => { Out.println(v); },
        Err(_) => { Out.println("失敗"); }
    }
    return 0;
}
```

出力

```
8
42
```

REWIND のタスクは **協調的なスケジューラ** の上で動きます。OS のスレッドで並列に走るのではなく、`await` や `std.task.yieldNow()` などの切り替え点で順番に実行され、結果は毎回同じになります。

タスクに渡す引数は `Send` でなければなりません。変更できる値は `move` で渡すか、`freeze` した値を渡します。借用はタスクへ渡せません。

REWIND

```
async fn total(xs: List<Int>) -> Int effects {} {
    return xs.len();
}

fn main()->Int {
    let xs = [1, 2, 3];
    match await spawn total(move xs) {
        Ok(n) => { Out.println(n); },
        Err(_) => {}
    }
    return 0;
}
```

出力

```
3
```

14.2 失敗、取り消し、時間切れ

`TaskError` には、タスクの内側で起きた失敗 `Failed(診断)`、`cancel()` による `Cancelled`、`timeout(n)` による `TimedOut` があります。時間切れは実時間ではなく、タスクが実行した論理的なステップ数で数えます。

REWIND

```
async fn spin() -> Int effects {} {
    while true {}
    return 0;
}

async fn divide(a: Int, b: Int) -> Int effects {} {
    return a / b;
}

fn main()->Int {
    let t = spawn spin();
    t.cancel();
    match await t {
        Err(TaskError::Cancelled) => { Out.println("取り消し"); },
        _ => {}
    }

    match await spin().timeout(20) {
        Err(TaskError::TimedOut) => { Out.println("時間切れ"); },
        _ => {}
    }

    match await spawn divide(1, 0) {
        Err(TaskError::Failed(d)) => { Out.println(d.code); },
        _ => {}
    }
    return 0;
}
```

出力

```
取り消し
時間切れ
DivisionByZero
```

`t.isDone()` は待たずに完了を調べます。タスクの失敗の詳細は `std.taskError.describe` で調べられます。

14.3 チャネル

`Channel<T>(容量)` は、タスク間で値を受け渡す通路です。容量 0 は、送り手と受け手が出会うまで待ち合わせます。`send` と `receive` は `await` で待ち、`close()` で閉じます。閉じて空になると `receive` は `Err` を返します。

REWIND

```
async fn producer(c: Channel<Int>) -> Unit effects {tasks} {
    for i in 0..3 {
        match await c.send(i) { Ok(_) => {}, Err(_) => {} }
    }
    c.close();
}

async fn consumer(c: Channel<Int>) -> Int effects {tasks} {
    var total = 0;
    while true {
        match await c.receive() {
            Ok(v) => { total += v; },
            Err(_) => { break; }
        }
    }
    return total;
}

fn main()->Int effects {tasks, output} {
    let c = Channel<Int>(1);
    let p = spawn producer(c);
    let r = spawn consumer(c);
    match await r {
        Ok(v) => { Out.println(v); },
        Err(_) => {}
    }
    return 0;
}
```

出力

```
3
```

14.4 `TaskGroup`

`using group = TaskGroup();` で作ったグループに `group.add(タスク)` で加えたタスクは、グループのスコープを出るときに必ず片付けられます。タスクが残ったまま関数が終わることはありません。

- 正常にスコープを出るときは、加えたタスクが終わるまで **待ちます**。
- `Result` を返す関数が `Err` を返して抜けるときは、まだ終わっていないタスクを **取り消します**。

REWIND

```
async fn work(n: Int) -> Unit effects {output} {
    Out.println("work ${n}");
}

fn run() -> Unit effects {tasks, output} {
    using group = TaskGroup();
    group.add(work(1));
    group.add(work(2));
    Out.println("スコープの終わり");
}

fn main()->Int effects {tasks, output} {
    run();
    Out.println("戻った");
    return 0;
}
```

出力

```
スコープの終わり
work 1
work 2
戻った
```

終わらないタスクを待つと、そのタスクが実行予算を使い切って `TaskGroupFailed` になります。

なお、`spawn` したタスクを一度も `await` せずに `main` が戻ると、そのタスクは実行されないことがあります。結果を使うタスクは必ず `await` するか、`TaskGroup` に入れてください。

15. 後始末と組み込みの入出力

15.1 `defer`

`defer` は、現在のスコープを出るときに実行する処理を予約します。`return` で出るときも、失敗で出るときも実行され、予約したのと **逆の順** に実行されます。ループの本体のスコープでは、反復ごとに実行されます。

REWIND

```
fn work() -> Unit effects {output} {
    defer || -> Unit { Out.println("後始末 1"); };
    defer || -> Unit { Out.println("後始末 2"); };
    Out.println("作業");
}

fn main()->Int {
    work();
    return 0;
}
```

出力

```
作業
後始末 2
後始末 1
```

`defer 式;` の形で、式をそのまま予約することもできます。後始末も失敗したときは、元の失敗を主な診断として残し、後始末の失敗は原因（causes）に追加されます。

15.2 `using`

`using 名前 = 式;` は、資源（ファイルハンドル、データベース接続、`TaskGroup` など）を、スコープを出るときに確実に閉じます。`return` で出るときも、失敗で止まるときも閉じられ、`defer` で閉じる処理を書くのと同じ順序規則（予約の逆順）に従います。

REWIND

```
fn readTail() -> String effects {fileRead} {
    using f = File.openSnapshot("a.txt");
    f.seek(6);
    return f.read(5);
}

fn main()->Int effects {fileRead, fileWrite, output} {
    File.writeText("a.txt", "hello world");
    publish;
    Out.println(readTail());
    return 0;
}
```

出力

```
world
```

`File.openSnapshot(path)` は、開いた時点のファイルの内容を読むハンドルを返します。`seek(位置)` で位置を決め、`read(n)` で `n` 文字分を読みます。

15.3 組み込みの入出力

次の名前は、`import` なしで使えます。必要な効果は 第 12 章の表のとおりです。

呼び出し	戻り値と備考
`Out.println(値)` 、`Err.println(値)`	`Unit` 。標準出力・標準エラーへの保留中の出力
`Out.writeBytes(Bytes)`	`Unit` 。生のバイト列を出力する
`Out.flush()` 、`Err.flush()`	`Unit` 。公開とは別の操作
`In.readLine()`	`Option<String>` 。入力の終わりは `None`
`In.readSecretLine()`	`Option<Secret<String>>` 。値は診断や記録に残らない
`In.readChunk(n)`	`Option<Bytes>`
`Args.all()`	`List<String>` 。起動引数
`Env.get(名前)` 、`Env.getSecret(名前)`	`Option<String>` 、`Option<Secret<String>>`
`Locale.current()`	`String` 。地域設定
`Time.now()`	`Int` 。観測された時刻。型付きの日時は `std.datetime` 、`std.clock`
`Random.next()`	`Int` 。VM の乱数
`File.readText(path)` 、`File.readBytes(path)`	`Result<String,FileError>` 、`Result<Bytes,FileError>`
`File.writeText(path, 文字列)` 、`File.writeBytes(path, Bytes)`	`Unit` 。保留中の書き込み

呼び出し	戻り値と備考
`File.open(path)`	ファイルハンドル（`using` で閉じる）
`Directory.entries(path)`	`List<String>`

`Out.println` に渡す値は、`Int`、`Float`、`String`、`Bool`、`Option`、`Result`、`List` などを自由に出力できます。`struct` や `record` のような型の表示形式は実装の内部表現に近く、安定した形式ではありません。人が読む文字列は自分で組み立ててください（文字列補間）。

`Out.println` は改行付きで、改行なしの出力関数はありません。

`secret(値)` は値を `Secret<T>` で包み、`reveal(秘密)` で取り出します。`Secret` を出力すると `<redacted: Secret>` になります。診断や記録にも値は残りません。ただし暗号化や外部への送信の防止まで保証するものではありません。

15.4 文字列のメソッド

`String` の長さや位置の単位は、メソッドによって違います。

メソッド	単位・戻り値
`s.byteLen()`	`Int`。UTF-8 のバイト数
`s.charLen()`	`Int`。Unicode のスカラー値の数
`s.utf16Len()`	`Int`。UTF-16 のコード単位の数
`s.contains(t)` `s.startsWith(t)` `s.endsWith(t)`	`Bool`
`s.find(t)`	`Option<Int>`。**バイト位置**
`s.split(t)`	`List<String>`
`s.slice(a, b)`	`Result<String,String>`。**バイト位置** の半開区間。UTF-8 の境界でなければ `Err`
`s.parseInt()`、`s.parseFloat()`	`Result<Int,String>`、`Result<Float,String>`
`s.encodeUtf8()`	`Bytes`
`s.format()`	`String`（`Int`、`Float`、`Bool` にもある）
`b.decodeUtf8()`	`Result<String,String>`（`Bytes` のメソッド）
`b.slice(a, b)`、`b.byteLen()`	`Bytes` のバイト位置

REWIND

```
fn main()->Int {
    let s = "Hello, 世界";
    Out.println(s.byteLen());
    Out.println(s.charLen());
    Out.println(s.find("世"));
    Out.println(s.slice(0, 5));
    Out.println("12".parseInt());
    Out.println("x".parseInt());
    return 0;
}
```

出力

```
13
9
Some(7)
Ok(Hello)
Ok(12)
Err(InvalidIntOrOverflow)
```

Unicode のスカラー値は、人が見る「一文字」と一致するとは限りません（結合文字など）。文字単位の操作は `std.text`、一文字（書記素クラスタ）単位の操作は `std.unicode` を使います。

15.5 `List` と `Map` の組み込みメソッド

型	メソッド
`List<T>`	`add(x)`、`push(x)`、`set(i, x)`、`get(i)`、`len()`、`pop()`（`Option<T>`）、`iter()`
`Map<K,V>`	`set(k, v)`、`get(k)`（`Option<V>`）、`remove(k)`、`len()`、`keys()`

`get(i)` は範囲外のとき致命的な失敗になります。`contains`、`insert`、`clear` のような他の操作は、標準ライブラリの `std.collections`、`std.map`、`std.sequence` などにあります。

16. 実行予算と制限

REWIND のプログラムは、実行の量に上限を持ちます。無限ループや暴走した処理が、環境を使い切ることを防ぐためです。上限を超えると、実行は止まり、どの予算が尽きたかが診断に出ます。

16.1 予算の種類

予算	指定	既定値	尽きたときの診断
実行ステップ（全体の累計）	`--steps N`	1,000,000	`ExecutionBudgetExceeded`
ネイティブ処理量（全体の累計）	`--native-work N`	1,000,000	`NativeWorkBudgetExceeded`
1 タスクあたりのステップ	`--task-steps N`	100,000	`TaskBudgetExceeded`
履歴メモリ	`--history-memory 512MiB`	512 MiB	`HistoryMemory`
履歴ストレージ	`--history-storage 8GiB`	8 GiB	`HistoryStorage`
履歴の退避しきい値	`--spill-threshold 8MiB`	8 MiB	（超えた分をストレージへ退避する）

ステップは、プログラムが実行した命令の数です。**ネイティブ処理量**は、標準ライブラリがネイティブコードで行う処理（配列の演算、文字列の処理、`freeze` のコピーなど）の見積もりで、API ごとに費用が文書に書かれています。**履歴**は、チェックポイントが保持するデータです。

これらは「論理的な仕事量」の上限で、実際の CPU 時間やメモリ使用量そのものではありません。実行が重いと感じたら `rewind profile` で内訳を調べられます（第 17.5 節）。

予算は **累計** です。`revert` や `resume` で状態を戻しても、使った分は戻りません。このため、巻き戻しを繰り返すプログラムも、いつかは止まります。

エラーになる例

```
fn main()->Int {
    var i = 0;
    while true { i += 1; }
    return 0;
}
```

関数の呼び出しの深さにも上限があり、再帰が約 1,000 段を超えると `ExecutionBudgetExceeded: call depth` で止まります。

16.2 ソースの中で指定する: `runtime`

`runtime { 設定 = 値; ... }` を宣言の位置に書くと、プログラムの中から予算を指定できます。設定名は `executionSteps`、`historyMemory`、`historyStorage`、`spillThreshold` で、サイズには `KB`、`MB`、`GB`（`KiB`、`MiB`、`GiB`）を付けられます。

エラーになる例

```
runtime { executionSteps = 1000; }

fn main()->Int {
    var i = 0;
    while i < 100000 { i += 1; }
    return 0;
}
```

すでに使った累計の予算を、ソースの設定や `revert` で回復することはできません。

16.3 そのほかの制限

項目	上限
ソースファイル	1 MiB / module、module は 256 個まで
構文の入れ子	64 段
`freeze` の深さ	64 段（循環や資源を含む値は不可）
`Map` に入れる文字列のキーなど	各 API の文書を参照

Linux では、コンパイラと実行時にアドレス空間の上限（既定 2 GiB）が掛かり、`--memory-mib` で変えられます。Windows では OS の上限を提供しないため、`--memory-mib` を指定するとエラーになります。論理的な予算は両方の OS で有効です。OS がプロセスを強制終了した場合は、`Result` として回復できません。

17. 診断と記録

17.1 エラーの表示

エラーは、コード、位置、呼び出しの経路、対処のヒントの順に表示されます。

出力

```
error[DivisionByZero]: division by zero
  --> main.rw:2:12
  at divide (main.rw:2:12)
  at main (main.rw:6:12)
  at <entry> (main.rw:5:1)
  help: Check the divisor before division or remainder.
```

- 位置は `ファイル:行:列` で、どちらも 1 から数え、列は Unicode のスカラー値の数です（LSP へは UTF-16 の位置に変換して渡します）。
- `at` の行は、内側の呼び出しから外側へ向かって最大 32 件です。それ以上は省略されます。ローカル変数や引数の値は含まれません。
- ソースを含まない成果物（ `.rwc` ）でも、埋め込まれたファイル名と位置が出ます。
- 型検査のエラーは `error[type]` 、 `error[expected]` のようにカテゴリ名で表示され、実行時のエラーは下の表のようなコードが付きます。

コード	主な原因
`DivisionByZero`	整数を 0 で割った
`IntegerOverflow`	`Int` の範囲（符号付き 64 ビット）を超えた
`IndexOutOfBounds`	負の添字、または `len()` 以上の添字
`AssertionFailed` 、 `Panic`	`assert` 、 `assert_eq` 、 `panic`
`UseAfterMove`	`move` した後の値を使った
`MissingEffect`	効果の宣言または許可が足りない
`InvalidContinuation`	`revert` が戻れない保存点を指した
`ExternalBoundary`	`external` 領域で禁止されている操作をした
`ExecutionBudgetExceeded` 、 `NativeWorkBudgetExceeded` 、 `TaskBudgetExceeded`	実行予算の超過
`HistoryMemory` 、 `HistoryStorage`	履歴の予算の超過
`TaskGroupFailed` 、 `TaskCancelled`	タスクの失敗、取り消し

コード	主な原因
`ExternalStateConflict`	公開時に外部のファイルが変わっていた
`PublishPartiallyApplied`	公開が途中まで適用された

17.2 終了コード

終了コード	意味
0	正常終了
1〜63	`main` が返した値（診断に `ApplicationExit<値>` が出る）
64	コマンドの使い方の誤りなど、位置のない一般的な失敗
65	型・所有権の検査エラー、アサーションの失敗、実行時の失敗
70	`panic` 、効果の不足など、原因（causes）を伴う失敗
71	予算の超過
72	外部の状態の衝突（ `ExternalStateConflict` ）
73	公開が途中まで適用された
74	タスクの取り消し

17.3 機械可読な診断

任意のコマンドに `--diagnostic-format json` を付けると、標準エラーに JSON が出ます。標準出力はアプリケーションの出力だけです。トップレベルのキーは `exit_status` 、 `message` 、 `diagnostic` 、 `publish_failure` 、 `code` 、 `retryable` 、 `retry_hint` で、 `diagnostic` には `code` 、 `message` 、 `source` 、 `line` 、 `column` 、 `frames` 、 `hints` 、 `causes` 、 `task_id` 、 `wait_edges` があります。

17.4 記録と再生

`--record trace.json` を付けて実行すると、外から得た値（入力、時刻、乱数、ファイルの読み取り、外部領域の結果）、タスクの切り替えの選択、失敗の位置が記録されます。 `rewind replay trace.json --root DIR` は、同じ版のコンパイラで同じ実行を再現し、結果が記録と一致するか確かめます。再生では実際の時計や入力は使われません。

記録の方式	内容
`--record-mode debug`（既定）	1 命令ごとの状態の索引を持つ。 `rewind dap` で巻き戻しながらステップ実行できる。大きく、長い実行には向かない
`--record-mode compact`	命令数、実行順序のダイジェスト、観測、最終状態だけ。長い実行向き

秘密の値（ `Secret` 、 `--secret-env` 、 `In.readSecretLine()` ）は記録に残らないので、再生時に再度与える必要があることがあります。記録の形式はコンパイラの版をまたいで互換ではありません。版を上げたら記録し直してください。

17.5 仕事量の内訳

`rewind profile main.rw` は、関数ごと・行ごとの命令数とネイティブ処理量、チェックポイントが保持しているデータ、未実行の行（coverage）を報告します。時間ではなく、予算で数える論理的な仕事量です。既定の出力は JSON で、`--profile-format text` を付けると人が読む形式になります。

17.6 デバッガとエディタ

道具	使い方
デバッガ（DAP）	記録（debug 方式）から、逆実行を含むステップ実行ができる。debugger.md
LSP	`rewind lsp --root DIR`。診断、補完、ホバー、定義・参照、リネーム、整形に対応
互換性の確認	`rewind compat` は、コンパイラ、言語、成果物、記録、ロックファイルなどの版の関係を表示する

18. 標準ライブラリの概観

標準ライブラリの module は `import std.名前 as 名前;` で使います。公開される関数、型、費用、失敗条件は、module ごとの文書（SDK の `share/rewind/doc/std/`）と libraries/README.md にあります。ここでは分類だけを示します。

分類	module
値と変換	option、result、error、failures、issues、number、integer、modular、bigint、decimal、math、bits、bytes、units、layout
文字列とテキスト	text、unicode、textIndex、textEdit、regex、stringSearch、url、path
日時	datetime、clock
コレクションとアルゴリズム	collections、map、orderedMap、sequence、sort、search、heap、deque、bitset、disjointSet、rollbackSet、fenwick、segment、lazySegment、range、trie、suffix、dp、table
グラフと幾何	graph、graphLarge、graphLargeAsync、flow、matching、geometry
データ形式と解析	json、jsonStream、csv、csvStream、config、args、scanner、stream、parser、schema、logfmt、compress、image
履歴と状態	snapshot、history、historyGraph、timeline、changes、persistentSet、memo、lazy、lens
数値計算	numeric とその Async 系、matrix、sparse、sparseAsync、fft、fftAsync、distributions
機械学習	autodiff 系、optimize、training、models、evaluation、plot
タスクと通信	task、taskError、retry、external、http、httpHeaders、httpRouter、httpServer、tcp
データベース	db、dbDatetime、dbDecimal、dbMigration、dbTransaction
GUI	gui、guiDialog、guiField、guiForm、guiMenu、guiTable、guiWindows
テスト支援	proptest

Option と Result の補助には `std.option`、`std.result` があり、`valueOr` で失敗時の既定値を指定して取り出せます。

REWIND

```
import std.option as option;
import std.result as result;

fn main()->Int {
    let missing: Option<Int> = None;
    let failed: Result<Int, String> = Err("bad");
    Out.println(option.valueOr(missing, 7));
    Out.println(result.valueOr(failed, 8));
    return 0;
}
```

出力

```
7
8
```

付録 A. 構文の一覧

次は書き方の形です。大文字の語は説明用の記号で、そのまま実行できるプログラムではありません。

用途	形
束縛	`let 名前: 型 = 式;`、`var 名前 = 式;`、`let (a, b) = 式;`
定数	`const 名前: 型 = 式;`
代入	`名前 = 式;`、`名前 += 式;`（`-= *= /= %=` も）、`xs[i] = 式;`
分岐	`if 式 { } else if 式 { } else { }`、`if let パターン = 式 { }`
反復	`while 式 { }`、`for 名前 in 開始..終了 { }`、`for 名前 in 式 { }`
パターン分岐	`match 式 { パターン => 式, パターン if 条件 => { }, _ => { } }`
取り出しと脱出	`let パターン = 式 else { return ...; };`
関数	`fn 名前<T>(引数: 型, 引数: 型 = 既定値) -> 型 effects { } { }`、`fn 名前(...) -> 型 = 式;`
非同期関数	`async fn ... { }`
テスト	`test fn 名前() { }`
型の定義	`struct`、`record`、`enum`、`type 名前 = 型;`、`trait`、`impl トレイト for 型 { }`
module	`import 経路 as 別名;`、`import 経路.{名前, 名前 as 別名};`、`pub`
借用・移動	`&式`、`&mut 式`、`move 式`、`freeze(式)`、`thaw(式)`
クロージャ	`|引数: 型| -> 型 { 本体 }`
エラー	`式?`、`panic(文字列)`、`assert(条件)`、`assert_eq(a, b)`
チェックポイント	`commit 名;`、`revert 名;`、`resume 名;`、`drop 名;`、`branch 名 { }`、`publish;`
外部領域	`external { }`、`external fresh { }`、`external live { }`
後始末	`defer || -> Unit { };`、`defer 式;`、`using 名前 = 式;`
並行	`spawn 式`、`await 式`
予算	`runtime { 設定 = 値; }`

付録 B. 言語バージョンの違い

REWIND は、`rewind.toml` の `language` に書いた版の言語規則でプログラムを検査します。単体のファイルの実行では、コンパイラと同じ版（3.6.0）が使われます。

版	言語の変更
3.0.0	実行コードは関数の中だけに書く。`fn main` から始まる。ブロック式、`if` 式、`let ... else`、`if let`、コレクションのリテラル `[ ]` `{ : }`、添字、`.0` によるタプルの要素、文字列補間 `${}`、式本体の関数 `= 式`。`main` が戻ると自動で公開する。文字列の大小比較、`Float.floorChecked` / `roundChecked`、`.format()`
3.1.0〜3.5.0	構文の変更なし。ツール（診断、デバッガ、profile、CLI）と標準ライブラリの追加
3.6.0	名前付きの引数と既定値の引数

3.x では、`const` の宣言とマップのキーの `for` による反復が使えない不具合があり、この版で修正されました。2.0.0 以前の言語の仕様は、それぞれの版の文書（REWIND_v2.0.md など）にあります。版ごとの詳しい変更点は `docs/REWIND_v*.md` を参照してください。

付録 C. 既知の制約

実装の現状として、次の点は期待と異なることがあります。

- `?` のエラー型の不一致は、`Err` の経路を実際に通ったときに実行時エラーとして検出されます（第 10.2 節）。
- ジェネリックな関数の型制約に、存在しない名前（`T: Bogus`）を書いても拒否されません。
- `Out.println` が `struct` や `record` を表示する形式は内部表現に近く、安定していません。
- 公開されていない関数や型を別の module から使うと、`unknown name` と報告されます（「非公開」という説明は出ません）。
- `List` と `Map` を `==` や `assert_eq` で比べると、中身ではなく同一性を調べます。
- `branch` の中で `return` すると、関数からは戻れますが、`branch` が戻すはずの状態は戻りません。